

Rete neurale per la classificazione di aritmie cardiache su dataset MIT-BIH

Samuel Paoli

2026-06-15

INDICE

1	Introduzione	1
2	Le architetture	2
2.1	Architettura convoluzionale (CNN)	2
2.2	Architettura ricorrente (LSTM bidirezionale)	3
2.3	Architettura ibrida	4
3	Strategia di addestramento	5
3.1	Partizionamento dei dati e stratificazione	5
3.2	Gestione dello sbilanciamento	6
3.3	Controllo dinamico dell'addestramento	6
4	Analisi prestazionale	8
4.1	Analisi comparativa	8
4.2	Analisi del modello ottimale	9
5	Conclusione	10

1 Introduzione

L'analisi automatizzata dell'elettrocardiogramma (ECG) tramite deep learning offre delle buone potenzialità diagnostiche. L'allenamento è stato svolto con il dataset MIT-BIH, il quale è già stato elaborato isolando i battiti, normalizzandoli a segnali della stessa lunghezza e classificandoli. Le classi sono 5: normale (N), sopraventricolari (S), ventricolari (V), fusioni (F) e non classificabili (Q).

L'addestramento sui dati clinici, come il dataset pubblico MIT_BIH Ayrhythmia Database, presenta delle problematiche. Poichè la stragrande maggioranza dei battiti appartiene alla classe Normale (N), le reti neurali tendono ad ignorare le altre classi. In questo modo infatti il modello raggiunge valori molto alti di accuratezza classificando quasi tutto come normale, risultando però inutile a causa dei numerosi falsi negativi. Un altro problema è la similarità di alcune classi di battiti, come la N, la S e la F, aumentando la complessità del problema.

Questo progetto affronta tali vulnerabilità attraverso un approccio sperimentale comparativo. Nello specifico, sono stati implementati e confrontati tre diversi modelli di Deep Learning: una Rete Neurale Convoluzionale (CNN), una Rete Neurale Ricorrente (LSTM) e, infine, un'architettura ibrida (CRNN). Attraverso questi modelli, sono state testate due diverse strategie per la gestione dello sbilanciamento: una funzione di costo pesata (che penalizza in modo più severo gli errori sulle classi minoritarie) e un approccio di Data Augmentation dinamico, tramite il quale i battiti rari vengono sovracampionati in memoria per ottenere batch di addestramento perfettamente bilanciati. Infine, viene affrontato e mitigato il problema dell'overfitting tramite l'implementazione di tecniche di regolarizzazione e di arresto anticipato (Early Stopping).

2 Le architetture

Al fine di valutare il miglior modo per la classificazione dei segnali elettrocardiografici, sono state sviluppate e confrontate tre diverse architetture di Deep Learning. Poiché ogni battito è rappresentato da una serie monodimensionale di 187 campioni, i modelli sono progettati per processare sequenze 1D.

2.1 Architettura convoluzionale (CNN)

Le reti convoluzionali sono eccellenti nell'estrazione di pattern locali. Lo scopo di questa architettura è di scansionare il battito alla ricerca di "forme specifiche", senza tener conto di dove si trovino lungo l'asse temporale.

```
class conv_1d_net(nn.Module):
    def __init__(self, n_classes=5):
        super().__init__()

        self.features=nn.Sequential(
            nn.Conv1d(1, 32, 5),
            nn.ReLU(),
            nn.Dropout(),
            nn.MaxPool1d(2),
            nn.Conv1d(32, 64, 5),
            nn.ReLU(),
            nn.MaxPool1d(2)
        )

        self.classifier=nn.Sequential(
            nn.Dropout(),
```

```

        nn.Linear(2752, 128),
        nn.ReLU(),
        nn.Dropout(),
        nn.Linear(128, n_classes)
    )

    def forward(self, x):
        x=self.features(x)
        x=x.view(x.size(0), -1)
        out=self.classifier(x)

    return out

```

L'architettura è strutturata in due blocchi. Il primo (*features*) riceve in ingresso un tensore corrispondente al segnale di 187 campioni. Attraversando il primo layer convoluzionale, caratterizzato da un *kernel_size* pari a 5, la lunghezza del segnale si riduce a 183 campioni. L'uscita passa poi attraverso la funzione di attivazione *ReLU*, il cui scopo è di introdurre la non-linearità necessaria azzerando i valori negativi. Successivamente interviene la funzione *Dropout* che, spegnendo una percentuale di neuroni ad ogni iterazione, costringe la rete a non dipendere da percorsi specifici, distribuendo l'apprendimento su tutto il layer.

Il secondo layer convoluzionale è uguale al primo tranne che per la rimozione del *dropout*. In uscita da quest'ultimo stadio, il tensore ha una profondità di 64 canali ed una lunghezza di 43.

Il secondo blocco riceve in ingresso il tensore in uscita dal primo blocco appiattito, quindi di lunghezza pari a $43 \cdot 64 = 2752$. Questo blocco è composto da due strati fully connected, che infine proietta le caratteristiche estratte verso i 5 nodi di output corrispondenti alle 5 classi.

2.2 Architettura ricorrente (LSTM bidirezionale)

L'architettura ricorrente è progettata per valutare l'evoluzione temporale del segnale, elaborando il battito un istante alla volta. Nello specifico è stata utilizzata una rete LSTM in configurazione bidirezionale.

```

class Bidir_LSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_layers = 3
        self.n_hidden = 64
        self.n_classes = 5
        self.drop_prob = 0.5
        self.n_input = 1
        self.batch_size=64

        self.lstm1=nn.LSTM(self.n_input, int(self.n_hidden/2), self.n_layers, bidirectional=True, dropout=self.drop_prob)

        self.fc=nn.Linear(self.n_hidden, self.n_classes)

```

```
def forward(self, x):
    x=x.permute(0,2,1)
    x, (hn, cn)=self.lstm1(x)
    out_forward=hn[-2, :, :]
    out_backward=hn[-1, :, :]
    out=torch.cat((out_forward, out_backward), dim=1)
    out=self.fc(out)

    return out
```

Il blocco principale è costituito da 3 layer LSTM sovrapposti, con un livello di dropout impostato al 50% interposto fra di essi. Come detto la rete è impostata in configurazione bidirezionale, quindi il segnale non viene letto solo cronologicamente dal tempo iniziale al tempo finale, ma anche a ritroso. Questo permette al modello di contestualizzare ogni istante basandosi sia sugli eventi passati che su quelli futuri.

Per mantenere coerente la dimensione computazionale, la dimensione nascosta della singola direzione viene divisa a metà. In uscita si ottiene un tensore hn , il quale contiene le memorie di tutti i layer in entrambe le direzioni. Quindi, per la classificazione finale viene estratto lo stato nascosto dell'ultimo layer di entrambe le direzioni.

Questi due vettori da 32 elementi vengono infine concatenati assieme, ricostruendo un singolo vettore di dimensione 64. Tale vettore viene dato in ingresso allo strato lineare finale, che ne esegue la classificazione.

2.3 Architettura ibrida

Per superare i limiti dei due modelli precedentemente trattati, è stato creato un modello CRNN che unisce l'efficienza spaziale della CNN con la memoria temporale della LSTM. Con questo approccio la CNN agisce da filtro che estrae le informazioni principali e successivamente la LSTM elabora questa sequenza di informazioni molto più ridotta.

```
class CNN_LSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_layers = 4
        self.n_hidden = 64
        self.n_classes = 5
        self.drop_prob = 0.5
        self.n_input = 64
        self.batch_size=64

        self.CNN=nn.Sequential(
            nn.Conv1d(1, 32, 5),
            nn.ReLU(),
            nn.Dropout(),
            nn.MaxPool1d(2),
```

```

        nn.Conv1d(32, 64, 5),
        nn.ReLU(),
        nn.MaxPool1d(2)
    )

    self.lstm1=nn.LSTM(self.n_input, int(self.n_hidden/2), self.n_layers, bidirectional=True, dropout=self.dropout)

    self.fc=nn.Linear(self.n_hidden, self.n_classes)
def forward(self, x):
    x=self.CNN(x)
    x=x.permute(0,2,1)
    x, (hn, cn)=self.lstm1(x)
    out_forward=hn[-2, :, :]
    out_backward=hn[-1, :, :]
    out=torch.cat((out_forward, out_backward), dim=1)
    out=self.fc(out)
    return out

```

Come si evince dal codice, l'architettura è composta dai moduli precedentemente analizzati, con alcuni accorgimenti necessari. Innanzitutto, per permettere il collegamento tra i due blocchi, è stato necessario inserire la funzione `permute(0, 2, 1)`: questa inverte gli assi del tensore in uscita dalla CNN, rendendolo compatibile con l'input atteso dalla rete ricorrente. Inoltre, il parametro `n_input` della LSTM è stato adattato ai 64 canali in uscita dall'estrattore convoluzionale, e il numero di layer ricorrenti è stato aumentato da 3 a 4.

3 Strategia di addestramento

Per garantire che il modello apprenda delle feature generalizzabili e non si limiti ad imparare a memoria il dataset, è stata progettata una strategia di addestramento. In questa fase è stato affrontato il problema dello sbilanciamento del numero di campioni per ogni classe.

3.1 Partizionamento dei dati e stratificazione

Il dataset MIT-BIH è fornito originariamente di due subset distinti: uno di test e uno di train. Il set di test è stato mantenuto completamente isolato da quello di train e utilizzato esclusivamente per la valutazione finale, simulando così le prestazioni del modello con pazienti mai visti.

Tuttavia, per monitorare l'apprendimento del modello ed evitare l'overfitting, è stato estratto un *Validation set* dai dati di train. Questo nuovo subset, che corrisponde al 10% del totale, funge da banco di prova intermedio e viene perciò completamente isolato dalla fase di aggiornamento dei pesi. Nella fase di creazione di questo set è cruciale l'applicazione della stratificazione: data la rarità di alcune classi, se si fosse creato il *Validation set* estraendo una porzione casuale dal set di partenza, si sarebbe rischiato di escludere completamente alcune classi. La stratificazione assicura invece che le proporzioni percentuali originali delle 5 classi vengano mantenute in entrambi i sotto-insiemi.

3.2 Gestione dello sbilanciamento

Il problema principale del dataset MIT-BIH risiede nella sua naturale distribuzione, dove circa il 90% dei campioni appartiene alla categoria dei battiti normali (N). Per evitare il collasso della rete verso le classi maggioritarie sono stati implementati e confrontati due diversi metodi: la pesatura della funzione di costo e il data augmentation.

3.2.1 Funzione di costo pesata

```
classi, conteggi=np.unique(y_tr, return_counts=True)
pesi=len(y_tr)/(len(classi)*conteggi)

class_weights=torch.tensor(pesi, dtype=torch.float32).to(device)

loss_function=nn.CrossEntropyLoss(weight=class_weights)
```

Con questo metodo sono stati calcolati dei coefficienti di penalizzazione inversamente proporzionali alla frequenza di ciascuna classe nel set di train. Passando questo tensore di pesi alla funzione di loss, il gradiente viene penalizzato in modo asimmetrico: un errore di classificazione su una classe rara genera una modifica molto più severa e marcata dei parametri rispetto ad un errore commesso su un battito sano, forzando la rete a prestare maggiore attenzione alle anomalie.

3.2.2 Data augmentation

```
classi, conteggi=np.unique(y_tr, return_counts=True)
pesi_classi=1.0/conteggi
pesi_sing=[pesi_classi[int(etichetta)] for etichetta in y_tr]
tensore_pesi=torch.DoubleTensor(pesi_sing)
sampler_bilanciato=WeightedRandomSampler(weights=tensore_pesi, num_samples=len(tensore_pesi), replacement=True)

train_loader=DataLoader(train_dataset, batch_size=64, sampler=sampler_bilanciato)
```

Il secondo metodo agisce a monte della rete neurale, intervenendo direttamente sul meccanismo di alimentazione del *DataLoader* di PyTorch. Utilizzando la classe *WeightedRandomSampler*, è stata implementata una tecnica di sovracampionamento. Il campionatore assegna a ogni singolo esempio un peso inversamente proporzionale alla numerosità della sua classe di appartenenza; di conseguenza, la selezione dei battiti per comporre il batch di addestramento (da 64 campioni) non avviene in modo puramente casuale, ma privilegia le classi minoritarie. Questo meccanismo costringe il sistema a duplicare dinamicamente le anomalie rare e a sottocampionare i battiti sani, garantendo l'equilibrio statistico in ogni singola iterazione di calcolo.

3.3 Controllo dinamico dell'addestramento

L'ultima criticità affrontata riguarda la gestione della convergenza della rete. L'addestramento prolungato del modello lo espone all'overfitting, ovvero la condizione in cui la rete inizia a memorizzare il rumore statistico, perdendo la capacità di generalizzare sui dati di validazione. Per contrastare

questo fenomeno e garantire che i pesi salvati siano quelli ottimali, è stato implementato un controllo dinamico della validation loss.

```
# Inizializzazione ottimizzatori, fuori dal ciclo di training

optimizer=optim.Adam(model.parameters(), lr=0.001)
lr_scheduler=optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=4)

# Ciclo di training

for epoch in range(epochs):

    # [...] fase di forward, calcolo loss e backward pass [...]

    lr_scheduler.step(avg_val_loss) # Aggiornamento learning rate

    # Reset del contatore se viene trovato un modello con loss di validazione migliore

    if min_loss>avg_val_loss:
        torch.save(model.state_dict(), model_filepath)
        min_loss=avg_val_loss
        msg_save="Modello salvato"
        epoca_migl=epoch+1
        count=0

    count+=1

    # Interruzione forzata se la validation loss non migliora

    if count>10:
        print("Nessun miglioramento nelle ultime 10 epoche")
        print("Addestramento interrotto")
        break
```

Il primo livello di difesa è costituito dalla modulazione del tasso di apprendimento tramite il modulo *ReduceLROnPlateau*. Mantenere un learning rate elevato per tutto l'allenamento non permette all'ottimizzatore di convergere in maniera fluida al minimo globale. Perciò, lo scheduler monitora costantemente la validation loss e, se non migliora nell'arco di 4 epoche, diminuisce il tasso di apprendimento di un fattore di 0.1. Ciò permette alla rete di effettuare aggiornamenti dei pesi molto più fini, sbloccando eventuali situazioni di stallo.

La seconda tecnica utilizzata agisce come controllo di sicurezza finale tramite l'arresto anticipato. Per fare ciò viene monitorato l'andamento della validation loss, continuando a salvare il modello in corrispondenza della loss minima. Tuttavia, se nonostante la continua diminuzione del learning rate la validation loss non diminuisce per 10 epoche consecutive, il ciclo di addestramento viene forzatamente interrotto. Questa logica previene l'overfitting, preservando esclusivamente i parametri dell'epoca più performante in assoluto.

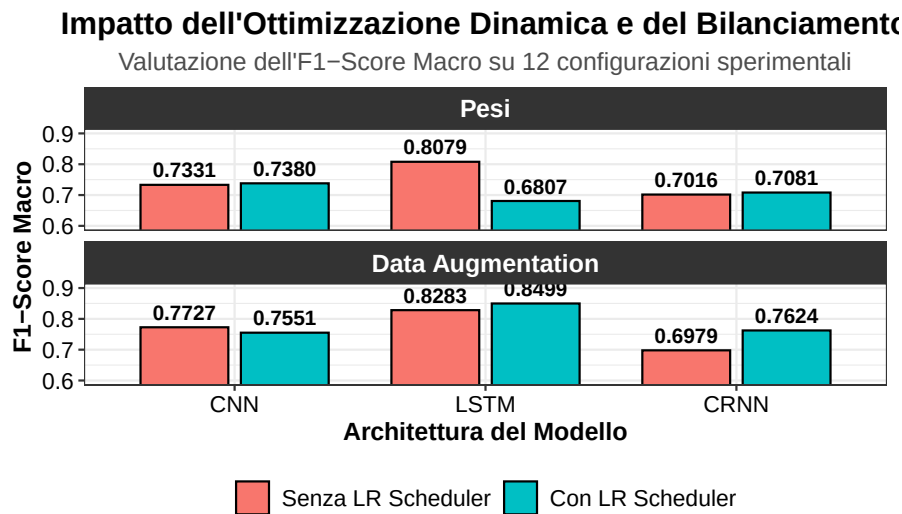
4 Analisi prestazionale

In questo capitolo vengono analizzati i risultati dei vari test svolti sui modelli creati. Data la natura sbilanciata del dataset di partenza, non è possibile utilizzare l'accuratezza globale per valutare le prestazioni dei vari modelli. Infatti se un modello classificasse ogni battito come normale avrebbe un'accuratezza del 90%, fallendo però nel suo scopo diagnostico.

Perciò, la valutazione comparativa dei modelli si basa sull'F1-score macro. Questa metrica si ottiene calcolando la media armonica tra Recall e Precision per ogni classe indipendentemente dalla sua numerosità, garantendo che le prestazioni sulle patologie rare abbiano lo stesso peso statistico della classe maggioritaria.

4.1 Analisi comparativa

Per quantificare il contributo di ogni tecnica adottata, sono state incrociate le tre architetture di rete con i metodi di gestione dello sbilanciamento e con le due strategie di ottimizzazione del gradiente. Il grafico seguente sintetizza i risultati delle 12 configurazioni testate.



In primo luogo, osservando il grafico, si evince chiaramente che con il metodo di data augmentation generalmente l'F1-score è più alto rispetto all'utilizzo di coefficienti di penalizzazione. Infatti, il sovracampionamento dei batch permette, soprattutto alle reti ricorrenti, di esporsi per un tempo maggiore alle geometrie delle patologie rare, migliorandone il riconoscimento.

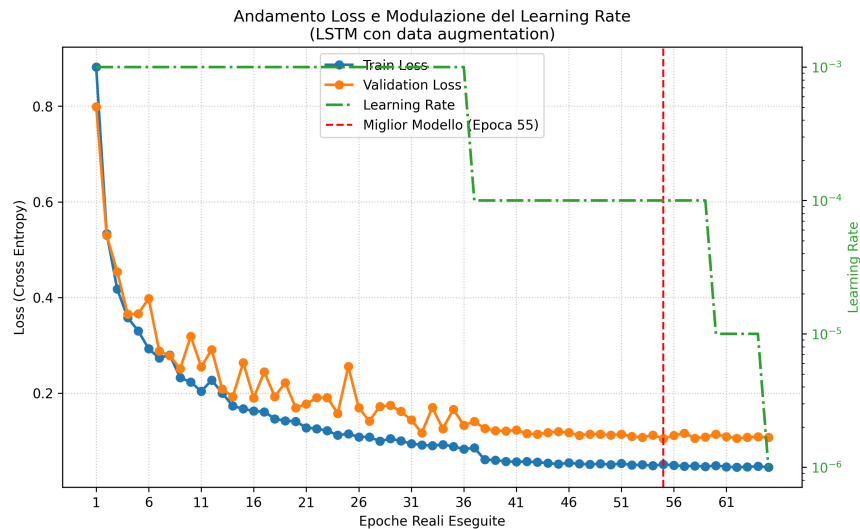
In secondo luogo, si osserva un impatto asimmetrico dello scheduler del learning rate. Si nota che nel caso della rete ibrida CRNN con data augmentation lo scheduler ha innalzato l'F1-score in maniera sostanziale, da 0.6979 a 0.7624. Al contrario, nell'architettura LSTM con i pesi, il taglio del tasso di apprendimento ha causato un deterioramento delle prestazioni, da 0.8079 a 0.6807. Questo fenomeno indica che la penalizzazione del gradiente indotta dai pesi, unita ad un tasso di apprendimento troppo basso, porta il modello a bloccarsi su minimi locali non ottimali.

Contrariamente alle aspettative teoriche che vedevano l'architettura ibrida CRNN come la più promettente, l'architettura LSTM con data augmentation e scheduler del learning rate ha registrato la prestazione migliore, con un F1-score di 0.8499. Tale risultato dimostra che, disponendo di sequenze temporali sufficientemente bilanciate, la LSTM possiede delle capacità di estrazione di informazioni di un segnale grezzo migliori rispetto ad un segnale pre-processato tramite convoluzione.

4.2 Analisi del modello ottimale

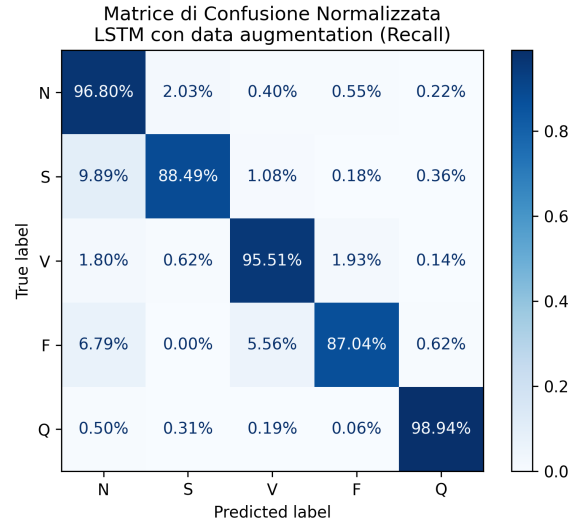
Per analizzare più nel dettaglio il modello con prestazioni più elevate, ovvero la rete LSTM, è opportuno valutare le dinamiche di addestramento e il comportamento predittivo.

In primo luogo, l'osservazione delle curve di apprendimento permette di validare la strategia di allenamento.



Il grafico illustra l'andamento della funzione di costo sui set di allenamento e validazione, sovrapposto alle variazioni del tasso di apprendimento. E' possibile notare che attorno all'epoca 37, la validation loss inizia ad appiattirsi. Tuttavia, tramite la diminuzione della learning rate, la curva ricomincia a scendere anche se non in maniera molto marcata. Successivamente si nota il meccanismo di early stopping che blocca la simulazione all'epoca 65, registrando i pesi dell'epoca 55.

Infine, per valutare il comportamento predittivo della rete, è stata generata la matrice di confusione normalizzata, strumento fondamentale per comprendere a fondo le dinamiche decisionali della rete.



Osservando la matrice di confusione e più nello specifico la sua diagonale principale, si nota che il modello riesce a classificare le classi N, V e Q in modo molto preciso, con un tasso di corretta identificazione rispettivamente di 96.80%, 95.51% e 98.94%. Il risultato più interessante, tuttavia, risiede nella capacità della rete di identificare con una buona precisione le patologie S e F. Queste due categorie, infatti, risultano particolarmente difficili da classificare in quanto sono molto simili alla categoria N. Nonostante ciò si è raggiunta una sensibilità rispettivamente di 88.49% e 87.04%.

Analizzando infine le percentuali esterne alla diagonale, si evince che le categorie S e F vengono frequentemente scambiate per battiti sani, rispettivamente nel 9.89% e nel 6.79% dei casi. Questi valori di falso negativo si mantengono comunque piuttosto bassi, dimostrando l'efficacia della rete.

5 Conclusione

L'obiettivo principale di questo progetto è stato lo sviluppo, l'ottimizzazione e il confronto di architetture di Deep Learning per la classificazione automatica di anomalie cardiache partendo da segnali elettrocardiografici monodimensionali. Per contrastare il marcato sbilanciamento del numero di campioni nelle varie classi sono state implementate specifiche tecniche di bilanciamento dei dati, affiancate da strategie di ottimizzazione e regolarizzazione il cui fine è stato prevenire il fenomeno dell'overfitting.

A seguito dell'analisi sperimentale svolta, si è giunti alla conclusione che il modello più performante è la rete LSTM Bidirezionale supportata da Data Augmentation e modulazione dinamica del tasso di apprendimento. Garantendo una sensibilità costantemente superiore all'87% anche nell'identificazione delle patologie più rare, il sistema così ottenuto si configura come uno strumento robusto e preciso.